

Regular Tekton Users (Pipelines-as-Code + Bitbucket)

UConn Private Container Registry (Quay) — Quick How-To for Pulling Images in Tekton and Deployments

Who this is for: Teams using Tekton Pipelines-as-Code (Bitbucket) who **pull** images from Quay (most common). If your pipeline also needs to **push** to Quay, see “When you need push access” below.

What’s already set up for you

- Quay org: matches your team name (same as namespace prefix).
- Standard repos: dev, test, prod (created by admins).
- Robot accounts: repo-scoped for push (admins distribute tokens).
- OpenShift imagePullSecret: created by onboarding automation for cluster pulls.
- Your pipeline ServiceAccount now includes the Quay pull secret, so Tekton pods can pull Quay images without extra per-pipeline wiring.

1) Sign in and confirm your repos

Registry: <https://quay.apps.uconn.edu> (Sign in with UConn EntraID).

In your org (e.g., its-ssg), you should see: dev, test, prod.

Note: regular users typically can’t view robot accounts in the UI; admins manage robot credentials.

2) Confirm Tekton can pull from Quay (pipeline ServiceAccount)

In each namespace (team-dev / team-test / team), the automation creates a pull secret named:

quay-pull-<namespace>

We’ve also attached that secret to the Tekton pipeline ServiceAccount so TaskRuns can pull images from Quay.

Verify (example for dev namespace):

```
oc -n <namespace> get sa pipeline -o
jsonpath='{.imagePullSecrets[*].name}{"\n"}'
oc -n <namespace> describe sa pipeline | egrep -i 'Image pull secrets|quay-
pull'
```

You should see a secret like quay-pull-<team>. If it’s missing, email aap@uconn.edu.

3) Use Quay-hosted images in Tekton Tasks

Once the pipeline ServiceAccount has the pull secret, you can reference Quay images directly in Task steps:

steps:

```
- name: run-tests
  image: quay.apps.uconn.edu/<org>/dev:<your-tooling-tag>
  script: |
    set -euo pipefail
    ./run-tests.sh
```

No additional registry credentials are required for pulls when using the pipeline ServiceAccount.

4) Pipelines-as-Code + Bitbucket secrets (source + GitOps)

Your Pipeline examples use secrets for Bitbucket auth (SSH or username/password) and for GitOps updates.

Keep Quay credentials separate from Git credentials:

- Bitbucket app/source auth: stored in a Secret (e.g., bitbucketsvc or bitbucket-ssh-auth).
- GitOps write auth: stored in a Secret (e.g., deploy-argo-tekton-write), mounted at /creds (as in your TaskRun).
- Quay pull access: comes from imagePullSecrets on the ServiceAccount (now including pipeline).

5) Using Quay images in deployments (runtime pulls)

When your Deployment references Quay, the namespace ServiceAccounts (default/builder/deployer, and now pipeline) can pull using the onboarded secret.

Example image reference:

```
image: quay.apps.uconn.edu/<org>/dev:myapp_v1.0
```

When you need push access (CI builds pushing to Quay)

Only robot accounts can push. If your Tekton pipeline must push to Quay:

- Request the correct robot username + token from your org admin / Platform Engineering (scoped to the repo you are pushing to).
- Store that token as a Secret in your namespace (do not store in Git).
- Reference it as a registry credential (e.g., dockerconfigjson or .dockercfg) in the Task that performs the push.

If you want, share your build-and-push Task (Kaniko/Buildah/S2I), and we'll provide a copy/paste Secret + mount layout that matches your standard.

Troubleshooting checklist

- 401 from Quay pulls: confirm the ServiceAccount used by the TaskRun has quay-pull-<team> in imagePullSecrets.
- Repo not found: repos are created by admins (dev/test/prod). Ask Platform Engineering for new repo requests.
- TLS errors: ensure your cluster/node trust bundle includes the registry CA (Platform Engineering can help).

Help: Platform Engineering (abiodun@uconn.edu, bharani@uconn.edu)